Benedikt Görgei

Connecting UAVs and Web Services using the Godot Video Game Engine and WebRTC to create a Meteaverse of Things

Bachelor Thesis
to achieve the university degree of
Bachelor of Science
Bachelor's degree programme: Electrical Engineering

submitted to

Graz University of Technology

Supervisor
Dipl.-Ing. BSc Markus Schuß
Assoc.Prof. Dott. Dott. mag. Dr.techn. Carlo Alberto Boano

Institute of Technical Informatics
Head: Univ.-Prof. Dipl-Inform. Dr.sc.ETH Kay Uwe Romer

Graz,October 27, 2022

AFFIDAVIT

I declare that I have authored this thesis independently, that I have not used other than the declared sources/resources, and that I have explicitly indicated all material which has been quoted either literally or by content from the sources used. The text document uploaded to TUGRAZonline is identical to the present master‘s thesis dissertation.

Date

Signature

Kurzfassung

In der heutigen Zeit, ist das Internet der Dinge ein essentieller Bestandteil unserer Internet Architektur geworden. Eine breite Vielfalt an sogenannten Smart Objects sind miteinander verbunden. Die Masse an Anwendungen reicht von Smart Home bis hin zu industriellen automatisierungs Prozessen und führt zu Herausforderungen im Bezug auf die Interoperabilität zwischen den einzelnen IoT Teilnehmern. Konsumenten müssen sich entscheiden welche Anwendung sie für sich und ihre Zwecke auswählen wollen und erhalten somit ein Set IoT-Services, die untereinander nicht zwangsläufig kompatibel sind.

In diesem Sinne sollte es eine Plattform geben, einen einzigen Markt, die eine maximale Anzahl an IoT Geräten in sich vereinen kann und gleichzeitig eine anwenderfreundliche Benutzererfahrung zulässt. Ein Metaverse of Things (MoT), welches auf bereits etablierten Standards aufbaut könnte als solche Plattform dienen. Der Beitrag dieser Arbeit ist es eine Definition und Prototypen eines simplen open source Metaverse of Things zu liefern, dass einen Drohnen Service für den Nutzer anbietet.

Abstract

Nowadys, the Internet of Things (IoT) has become an established part of the Internet architecture. A wide variety of so called smart objects are interconnect. The broad amount of applications ranging from *Smart Home* to industrial automatization processes leads to interoperability challenges. Consumers have to choose between several available services acquiring a set of apps to satisfy their needs.

In this regard, there should be one platform, one single market that should be able to integrate a maximum range of IoT devices and provide a convenient user experience at the same time. A metaverse of things (MoT) that implements already established standards such as WebRTC and WebGL may serve as a platform for IoT services. The contribution of this thesis is to give a definition and build a basic open source Metaverse of Things with an drone as a service experience for the user.

Contents

1 Introduction **7**
1.1 Problem Statement . . . 7
1.2 Objective . . . 8
1.3 Outline . . . 9

2 Background and Related Work **9**
2.1 A typical Metaverse . . . 9
2.2 The Godot Engine . . . 11
2.2.1 Features and Functionalities . . . 11
2.2.2 Tree, Node and Scene Structure . . . 11
2.2.3 Scripts . . . 12
2.2.4 Networking . . . 13
2.3 Home Network Server . . . 16
2.3.1 Hardware . . . 16
2.3.2 Web services with Nginx and Flask . . . 16
2.4 UAV . . . 17
2.4.1 Flight-computer: BeagleBoneBlue . . . 17
2.4.2 Hardware Components . . . 18
2.5 WebRTC (Web Real Time Communication) . . . 18

3 Implementation **20**
3.1 Webservice . . . 20
3.1.1 The Website and the Authentication Process . . . 20
3.2 Virtual Environment . . . 21
3.2.1 Lobby . . . 21
3.2.2 Connection . . . 21
3.2.3 3 Dimensional Space . . . 23
3.3 UAV Setup . . . 25
3.3.1 Flight Control Software . . . 25
3.3.2 The Broker and the MAVLINK Protocol . . . 25

4 Evaluation **26**
4.1 Latency of data packages . . . 26
4.1.1 Master and Observer in Local Network . . . 26
4.2 Master and Observer in separate Networks . . . 27
4.3 Master and UAV in Local Network . . . 28
4.4 Master and UAV in separate Networks . . . 29

5 Limitations **31**

6 Future Work **31**
6.1 Fixes and Changes . . . 31
6.2 Interoperability with the help of Web of Things . . . 32

7 Conclusion **32**

List of Figures

1 Editor scene view of the user avatar . . . 13
2 Game.tscn scene tree . . . 14
3 Inspector of user node . . . 14
4 Raspberry PI 4 . . . 16
5 BeagleBoneBlue . . . 17
6 WebRTC Schematic . . . 19
7 Network architecture overview . . . 21
8 Login page . . . 22
9 Iframe cut out from the master home page . . . 22
10 Sequence diagram of the connection process . . . 23
11 Virtual environment with 3D Model UAV, Host (red) and Client (green) avatar . . . 24
12 Gathered ICE candidates for local connection . . . 26
13 currentRoundTripTime for local connection among other selected statistical graphs . . . 27
14 Gathered ICE candidates for remote connection . . . 27
15 currentRoundTripTime for remote connection among other selected statistical graphs . . . 28
16 currentRoundTripTime for remote UAV connection among other selected statistical graphs . . . 28
17 Gathered ICE candidates for local UAV connection . . . 29
18 Gathered ICE candidates for local UAV connection, table beginning . . . 29
19 Gathered ICE candidates for local UAV connection, table end . . . 30
20 currentRoundTripTime for remote UAV connection among other selected statistical graphs . . . 30

List of Tables

1 List of used features Godot features for the MoT. Further information can be found at [11]. 12

1 Introduction

On 28 October 2021 the company *Facebook, Inc.* rebranded its name to *Meta* and announced to create its own virtual reality world on a social economic basis. *Metaverse* was the major buzzword for this transition. The term *metaverse* has its origin in the science fiction novel "Snow Crash" by Neal Stephenson from 1992. In "Snow Crash" users access a virtual-reality with VR-goggles as avatars to socially interact with each other or trade virtual goods in a market. With Second Life by Linden Lab one of the first metaverse worlds launched in 2003 [1]. Hundreds of users can access and populate one persistent virtual environment. Other examples are [2] [3].

In the real world there is a technological shift driven by the Internet of Things (IoT). IoT interconnects physical devices called smart objects, collects, analyses and shares data. Computation and communication are the primary technological requirements for smart objects in the IoT . Many smart application in various fields have emerged in recent decades like smart homes, smart agriculture, industry 4.0, smart healthcare, environmental and industrial monitoring etc. However, aspects like scalability, security, reliability, interoperability, energy-efficiency makes the development IoT systems a challenge in practice [4].

When robotic and mobile devices become part of the IoT environment some smart objects go beyond pure data collection and communication . A good example are unmanned area vehicles (UAVs). They are used for surveillance of infrastructure and farm land as well as for search and rescue operation among other applications. Equipped with accelerometers, gyroscopes and other various sensors they do not only collect data, but also fulfill tasks either by direct control or autonomously. Prominently, Amazon suggests even using drones for autonomous delivery services [5]. UAVs also have received high attention from society and research communities. In general more and more autonomous smart objects are in development to increase quality of service of already established infrastructure and processes.

1.1 Problem Statement

All interconnection and data collection is useless, if it is not for the benefit of society in an economic way. There is an app for almost anything from smart-home to smart-health, different manufactures come up with different solutions. Factorization of collected data in too many hubs, services and tightly locked implementations limits efficiency and slows down development progress. The consumer of smart applications is in the inconvenient situation to choose one or the other smart service taking different approaches to solve the same problem. Interoperability of services from different providers is not guaranteed. On a service level, a metaverse that brings together people and machines in one persistent virtual environment might solve the issue of vendor locked applications as an intermediate platform.

There is currently no large scale metavers that brings real-time data from smart object into a digital reality. The data should not only be visible in a meaningful way, but also responsive to user inputs. A metaverse includes the interaction between humans, a meta-

verse of things (MoT) includes the interaction between humans and smart object in an virtual environment populated by digital twins and avatars of their real-life counterparts. It needs to be accessible always and anywhere in the world trough the Internet. Although, there is plenty of software available for UAV control not many of they try to close the gap between custom control apps and modern 3D rendering engines. [6]

1.2 Objective

In this thesis a MoT is suggested that aims to show the possibility of creating a simplistic, easy to use, human readable 3D interface that provides a drone as a service experience. It combines three major aspects:

- Information aggregation
  - Collect telemetry and first person view (FPV) camera data from the UAV
  - Store data about the type and the IP addresses of connected peers
  - Distribute collected state information from the UAV to all connected consumer peers
- Collaborative awareness
  - The UAV is able to perform simple tasks such as take-off, hold position, fly to a way-point on received commands with or without active control input
  - State change of the UAV is visually represented in an virtual environment
  - Control permissions of the UAV can be granted or withdrawn from users by one authorized user
  - Permissioned users are able to send commands to the UAV to change its state
- Ubiquitousness
  - Deployment of the service on a secured website providing a single access point for users

The goal is to implement the bare minimum of functionalities to create the suggested service and design an software architecture that at least creates the illusion of a real-time system for the user. Open source software shall be preferable used for any kind of implementation. Advanced topics about drone controls like localization, GPS-navigation or object detection are not covered. In this thesis the necessary steps of combining a 3D virtual environment with the IoT devices are explored. Therefore, the system is evaluated based on the suitability of all used software components in terms of interoperability, scale ability, real-time performance and whether or not the mentioned requirements are meet. In the end this thesis is a contribution to buid an open metaverse [7] and large scale IoT networks, to ever closer connect humans, machines, services and virtual environments.

1.3 Outline

Chapter 2 provides a technical overview. Related work regarding MoT as well as key concepts of the used video game engine are discussed. In addition a brief overview of the used hardware is presented. In Chapter 3 the architectural approach based on micro services is presented. All functionalities are explained in detail and a user story gives a walk through of an hypothetical use case. The performance of data transmission and is examined in chapter 4. Limitations to the presented work is discussed in chapter 5. Follow up projects and promising perspectives are presented in chapter 6. Finally, chapter 7 summarizes the results and findings and gives a verdict on the prospect of combining the metaverse concept with the IoT.

2 Background and Related Work

For the MoT one first have to identify the necessary software and hardware components involved. On the software side there is the virtual environment, the framework for the website and the IoT broker. On the hardware side there is the drone with its flight-computer and peripherals, web-server machine and consumer devices such as PCs, laptops or smartphones.

The virtual environment is the canvas for avatars and virtual twins of IoT objects and their states. Video game engines are attractive for creating interactive 3D environments. Since the MoT will include a drone service one could consider alternatives like the FlightGear flight simulator or specialized drone control apps. However, these solutions are application specific and are limited in terms of network capabilities. Game engines are proven to be versatile and near real-time capable, at least from a human perspective. Lots of players can populating the same multiplayer game and distribute their current status throughout the game network. Deployment to multiple platforms is another advantage. Technologies like WebGL allow the integration of video games seamlessly into websites. Plenty of web frameworks are available to create secure web applications. One broker is needed to manage all messages that pass between the drone and consumer devices. Its acts as a bridge between the web server and the UAV.

2.1 A typical Metaverse

The term metaverse has its origin in science fiction. The boundaries are hard to define and often compared to destopic copies of reality in movies like The Matrix, Ready Player One, Tron, or to massive multiplayer online games (MMOs). There is no broader consensus on what exactly a metaverse really is and definitions vary upon who is asked, but one can agree on some core attributes that most descriptions of a metaverse have in common:

- Persistent
  - Only one version of metaverse exists. All avatars populate the same virtual world. Changes to it apply to all users in the same way.
- Centralized

  - One service provider hosts and maintains the metaverse and has the ultimative control over it (Although, there have been suggestions for an open metaverse operating system with defined standards and protocols for metaverse-web-browsers and 3D rendering).

- Virtual Environment
  - Users are represented as avatars, which are basically visual representations of user accounts and they are usually customizable. The avatars populate a 3 dimensional handcrafted virtual world.

- Synchronus
  - Entities share information about the state of the virtual worlds and stream this data to people's computers with low enough latency that they don't even notice. Cloud technology is involved in streaming data. This includes avatars as well as assets possessed by the users or active world elements. Some metaverses are using blockchain technology to clearly identify ownership of virtual assets.

- Human focused experience
  - Metaverses are built around the concept of providing an alternative virtual reality for humans to interact with.

- Economy
  - Individuals buy, sell and own virtual real estate, goods, avatar adjustments etc. Content is created and sold by independent creators or commercial enterprises on dedicated markets inside the metaverse. The virtual values are separated from anything in real life.

Virtual environments are also known for many industrial sectors. They are used in the automotive, manufacturing, construction, military and avionics industries. Although, the focus is on simulating behavior for multiple processes in application scenarios through virtual models of physical objects. These so-called digital twins are meant to be exact virtual models of their physical counterparts in terms of appearance and properties based on mathematical frameworks [8].

Why should we even bother?
Metaverses as they are seem to be pure consumer and entertainment products. Their economic systems create value through the "Greater Fool" theory that one can make money through the purchase of overvalued assets, if these assets can be later sold at even higher price. Their markets are more like virtual accessory stores where people buy articles just to possess some piece of virtual art. The disappearance of all value in metaverses would not have a significant impact on society, because there is no interconnection to any economic infrastructure process in the real world. Breaking separation between virtual reality and the real world through the introduction of smart objects could potentially change the market system to a more involved one. The MoT would be offering data and IoT services similar to Netflix offering films and series on one platform. A possible application scenario could be

the sharing of UAV control and data collection in urban farming. One consumer purchased a drone service at the MoT market that identifies pest plants. From another company he purchases a service robot that cuts down the pest plants and control both devices with the MoT interface. Some video games like the Microsoft Fight simulator already use large amounts of satellite data and air plane signatures to create a detailed virtual copy of the earth with accurate real-time air plane positioning and weather conditions. The MoT would be the sink of all IoT services and data.

2.2 The Godot Engine

Godot Engine is an open source multi-platform game engine for creating 2D and 3D computer games released under the MIT License. It was created by Juan Linietsky and Ariel Manzur, two Argentine developers as a set of tools for clients in the game industry like Square Enix or Turner [9]. It is named after the play *"waiting for Godot"*. The Godot Engine has been chosen for MoT, because of its open source non-profit philosophy, community support, excellent documentation and extension capabilities for follow up projects. Most importantly it provides all the necessary tools and functionalities such as 3D rendering, a high level networking API, a WebGL web build etc. to create a metaverse [10].

2.2.1 Features and Functionalities

The Engine is programmed in C++. However, programming is supported in C, Visual Scripting, GDNative and GDScript a python like language unique to Godot. The Visual Editor is completely written in GDScript. Additionally, C++ Modules allow extending the engine in a modular way. New modules can be created, or external libraries bound to Godot. Unlike modules, GDNative can be used to run C or C++ code without compiling the engine's source code in a Godot project and allows to interact with native shared libraries at run-time. For MoT GDScript is used to reduce and simplify coding and debugging effort to create a 3D virtual environment. It also provides easy access to the Engines High Level Network API as explained in section 2.2.4 .

2.2.2 Tree, Node and Scene Structure

A project in Godot is structured in a tree consisting of nodes grouped together in reusable scenes. Nodes are the building blocks that are arranged into trees. The node types which can be added are predefined by the editor. In Figure 11 for example, the user avatar scene created for the virtual environment has a *KinematicBody* as its top node. It is subject to physics calculations such as collisions with other shapes and requires a CollisionShape node. If there are no dependencies the nodes can be arranged at will. Together with the 3DLable *Spatial*, *Tween* and NetworkTickRate *Timer* node the tree is saved as a scene that can be instanced in other scenes creating nested scene structures. Child nodes inherit the spatial position of parent nodes and spatial transformation is relative to the parent nodes global position in the scene. Every property of a node can be accessed in the inspector or via code, although the syntax to do so might not be coherent for all properties. The route scene for MoT environment is stored as Game.tscn and is accessed first on execution.

Feature	Used Feature
Platforms	- HTML5 via WebAssembly (Firefox, Chrome, Edge, Opera)
Editor	- Scene tree editor - Built-in script editor and debugger - Performance monitoring tools
Import	- 3D Mesh of the drone imported as Wavefront OBJ
- 3D tools and physics	Built-in meshes: cube, plane, shpere - Ray-cast on mouse click - Collision detection
Navigation	- A^* algorithm for path-finding - Navigation meshes
GUI System	- Menu Containers - Interactive Elements: (Check-)Buttons, Check-boxes, LineEdit, Radio-Buttos, Scrollbars
Networking	- High-level multiplayer API using UDP and ENet - Automatic replication using remote procedure calls (RPCs) - Low-level UDP networking using PacketPeer and UDPServer - WebSocket client to send data to send data the IoT Broker
Input	- Keyboard - Mouse - Touchscreen on android devices
Windowing and OS integration	- Move, resize, minimize, and maximize the window - Fullscreen mode
Shaders	- Text-based shaders used for highlighting
Scripting	- GDScript
File formats	- JSON including control commands for the UAV

Table 1: List of used features Godot features for the MoT. Further information can be found at [11].

2.2.3 Scripts

Every piece of code in Godot is a written in scripts attached to nodes (visible as white scrolls next to nodes). Scripts add behavior to nodes and define how it interacts with other nodes. Video games run in loops and Godot provides two virtual methods to update the state of the game `_process` and `_physics_process`. The first method is used for

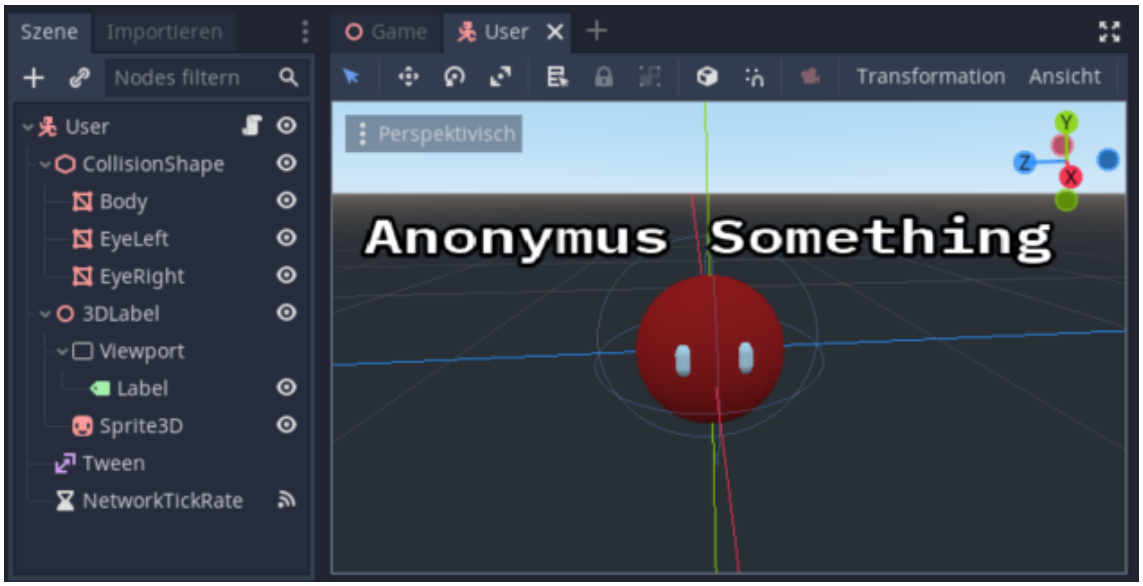

Figure 1: Editor scene view of the user avatar

idle processing and updates the node every frame, as often as possible. The latter is independent of the actual frame rate on the screen usually used for physic collisions and movements. Another one is `_ready` that is called first when the attached node enters the scene tree. To react to certain events, let's say a mouse click, some nodes have specific signals that can be connected to any function in a script. The function will be executed if the signal fires. In addition custom signals can be defined.

```
#Example script that rotates a spatial node
extends Spatial

func _ready():
    connect("clicked", self, "_on_mouse_click")

func _process(delta):
    #rotates the node along the x-axis with constant speed
    #delta is the time passed between two execution cycles
    rotate_object_local(Vector3(1, 0, 0), 1*delta)

func _on_mouse_click:
    rotation_degree.y += 10
```

2.2.4 Networking

In Godot one can distinguish between low-level networking and high-level networking. One the one hand, low-level means the use of an dedicated API called *PacketPeer* that itself accesses protocols like TCP, UDP, SSL and HTTP. This allows for maximised control for the network. On the other hand, the high-level API is based on *SceneTree* basically the

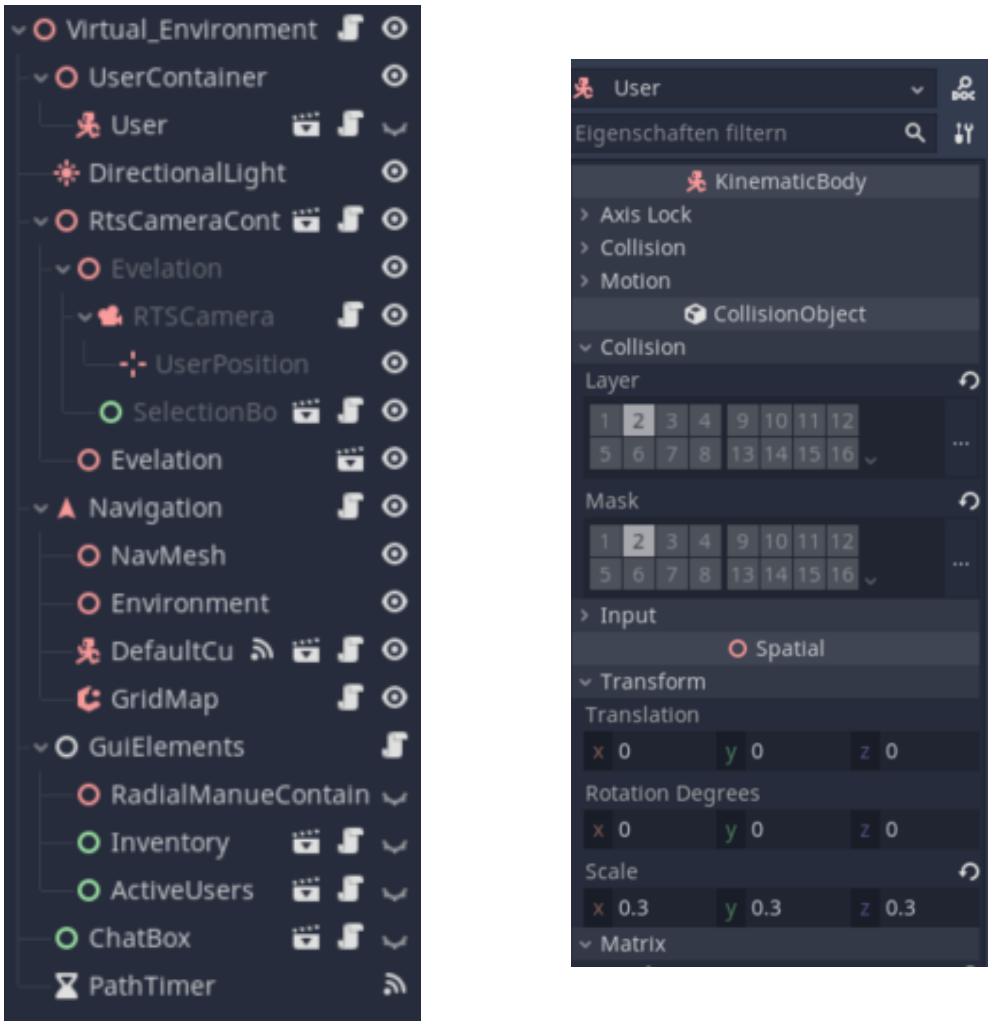

Figure 3: Inspector of user node

class that manages the hierarchy of nodes and scenes. An extension to *PacketPeer* is *NetworkedMultiplayerPeer* to provide methods to set a peer, transfer mode, etc. and provides signals for connection status checks. An implementation is provided based on ENet [12], WebRTC [13] and WebSocket [14].

A high-level network implementation based on ENet could look as follows: First the *SceneTree* must be provided with an NetworkedMultiplayerENet object that is first initialized either as a server or a client. Whether the current game is a client or a server can be chosen by accessing the according function `create_server()` or `join_server()` in the game lobby by the press of a button. When the node enters the scene tree the code in the `_ready()` function is executed and the default *SceneTree* signal "connected_to_server" will be connected with the custom function `_connected_to_server()`. This way the state of the connection can be monitored.

To share and synchronize information across a network it is necessary to access functions of other game instances remotely, using remote procedure calls (RPC). The keyword `remote` will allow an RPC to access this function via network. On top of that Godot makes use of a principle called *network master*. The network master of a node is the peer that has the ultimate authority over it. With `if is_network_master():` only the network master can execute the code section after this if statement. At default, the server is the network master for all nodes and it always has the network ID=1 and clients will have random generated unique ID numbers. If not otherwise defined the network ID of the game instance will be 1, but for the MoT project the ID is assigned by a signaling server (see chapter 3.2.2). This concept is quiet useful for synchronization multiple instances of objects across the network. With `set_network_master(id)` the authority over a node can be changed. For example: If a new user joins the server its id is registered and a user node is created with its authority set to the new registered id. With the keyword `puppet` property change of this new created node is only allowed for its network master, thus for its user. Update on position and orientation can be done by RPC. The condition to make this work is that all user nodes in all game instances are at the same position in the scene-tree.

```
extends KinematicBody

onready var timer = $NetworkTickRate

puppet var puppet_position  setget puppet_position_set
puppet var puppet_rotation  setget puppet_rotation_set

func puppet_position_set(new_value) -> void:
        global_transform.origin = new_value

func puppet_rotation_set(new_value) -> void:
        global_transform.basis = new_value

func _on_NetworkTickRate_timeout() -> void:
        if is_network_master():
            #RPC updates puppet variable values with argument
            rset_unreliable("puppet_position", Global.user_position)
            rset_unreliable("puppet_rotation", Global.user_rotation)
```

2.3 Home Network Server

2.3.1 Hardware

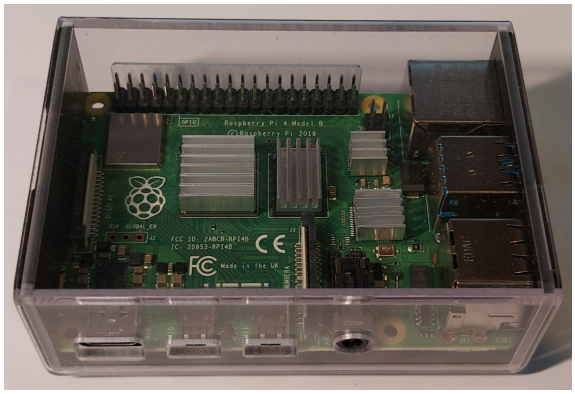

Figure 4: Raspberry PI 4

The famous Raspberry Pi is a highly supported Linux based minicomputer, mainly used for educational purposes. In the MoT the Raspberry Pi in its model 4 B variant is used as the web server that will hold three microservices. A website based on the flask python library, which will serve as the human access point to the network, a signaling server that handels managing of ids and connection establishment and the WebGL-build of the virtual environment that can be accessed on the website.

Component	Specification
Processor	Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
RAM	4GB LPDDR4-3200 SDRAM
Bluetooth	Bluetooth 5.0
Wi-Fi	2.4 GHz and 5.0 GHz IEEE 802.11ac wireless
USB	2 USB 3.0 ports; 2 USB 2.0 ports
Ethernet	Gigabit Ethernet
HDMI	2 × micro-HDMI ports
Storage	Micro-SD card slot for loading operating system and data storage
Power Supply	5V DC via USB-C connector; 5V DC via GPIO header
Dimensions	85.6mm × 56.5mm

2.3.2 Web services with Nginx and Flask

The website is created with Flask an python microframework for web development. Flask only depends on the Jinja template engine and Werkzeug WSGI toolkit. It is chosen due to simplistic core features and its suitability for extensions. It has the advantage of supporting REST APIs, which comes in handy to crate a clear defined interface.

2.4 UAV

2.4.1 Flight-computer: BeagleBoneBlue

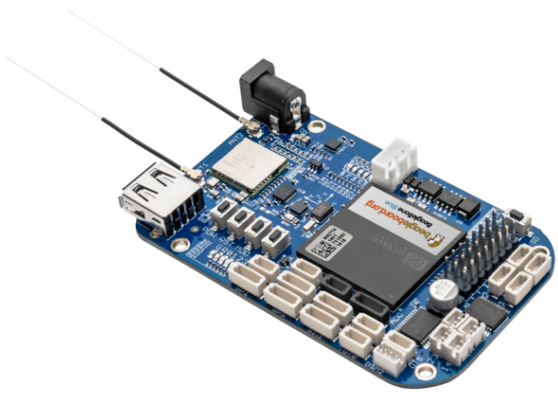

Figure 5: BeagleBoneBlue

The BeagleBoneBlue is a Linux-based mini-computer designed for educational robotic applications. In MoT project it is supposed to act as the flight-computer of the drone with the ardupilot copter 3.5 control software. The advantage of this board is that it comes with integrated sensors like a gyroscope and accelerometers that allow to stabilize the drone mid fight and has 8 servo output control outputs. The Debian operating system allow additional scripts to run in parallel with the control software. No additional companion board is needed to access the MAVLINK Interface or to establish an internet connection with Wi-Fi. As long as the board is connected to the internet, it can receive MAVLINK commands on port 14550 from authorized users via the home-network-broker and interprets them to control the drone.

Component	Specification
Processor	AM335x 1GHz ARM Cortex-A8 processor; 2×32-bit 200-MHz programmable real-time units (PRUs)
RAM	512MB DDR3 RAM
Bluetooth	Bluetooth 4.1 and BLE
Wi-Fi	Wireless: 802.11bgn
USB	USB2 client for power and communications
Sensors	9 axis IMU (accels, gyros, magnetometer), barometer, thermometer
Buses and peripherals	GPS, DSM2 radio, UARTs, SPI, I2C, 1.8V analog, 3.3V GPIOs
Storage	4GB 8-bit eMMC flash storage
Power Supply	Charger input: 9-18V; Battery support: 2-cell LiPo with balancing, LED state-of-charge monitor
Dimensions	88.9 × 54.61mm
Operating System	Debian Real Time OS

2.4.2 Hardware Components

The UAV is made up of a quad copter frame, four propeller motors controlled by ESCs. The ESCs are connected to the Beaglebone flight computer. The power source is a Lipo battery.

Component	Specification
Flight Computer	BeagleBoneBlue
Quadcopter Drone Frame	Nylon material
LiPo	2200mAh 11.1v 30c
4x propeller moters	1000 KV 28mm diameter
4x ESC	30A
USB web camera	1080p 30fps

2.5 WebRTC (Web Real Time Communication)

Since a metaverse should be accessible with ease by browsers, there is a need to exchange information between web-services and smart object native software. The WebRTC protocol adds real-time communication capabilities to an application. The technology is available on all modern browsers. First developed by Global IP Solutions WebRTC source code was published on 1st of June 2011 by Google. The WebRTC project is open-source and supported by Apple, Google, Microsoft and Mozilla, amongst others [15].

According to W3C, WebRTC 1.0 [16] covers the following facets:

- Connecting to remote peers using NAT-traversal technologies such as ICE, STUN, and TURN.
- Sending the locally-produced tracks to remote peers and receiving tracks from remote peers.
- Sending arbitrary data directly to remote peers.

The goal of WebRTC is to establish peer to peer connections between peers. In the current internet architecture peers may reside behind a NAT (Network Address Translation) device separating the peer from the internet in a local network with a private IP addresses. RFC 8489 STUN (Session Traversal Utilities for NAT) can be used by an endpoint to determine the IP address and port allocated to it by a NAT. Since a direct socket is not always possible, unless the clients reside in the same local network traffic needs to be relayed. RFC 5766 TURN (Traversal Using Relays around NAT) servers are common solutions. The TURN server acts as an intermediate node for communication relay. According to RCF 8445 ICE (Interactive Connectivity Establishment) both peers gather gather ICE host, server-reflexive, peer-reflexive and relayed candidates. The server-reflexive candidates are gathered using STUN or TURN, and relayed candidates are obtained through TURN. Host candidates are obtained by binding to ports on an IP address attached to an interface 18 on the host. Peer-reflexive candidates are gathered in a later phase as a consequence of connectivity checks. Each candidate is a potential address or port to receive

data. They are part of the RFC 8866 SDP (Session Description Protocol) in a WebRTC connection. Both peers need to exchange SDPs with the initializing peer sending an offer to one or more receiving peers with information about the type of session. The exchange happens through a signaling server with an IP address known to all participants. Since the initializing peer does not know how to reach the receiving peer jet. The signaling server forwards the offer and the receiving peer creates an SDP answer and sends it back again through the signaling server. If both peers are able to exchange data through the collected ICE candidates listed in the SDPs the connection is successfully established.

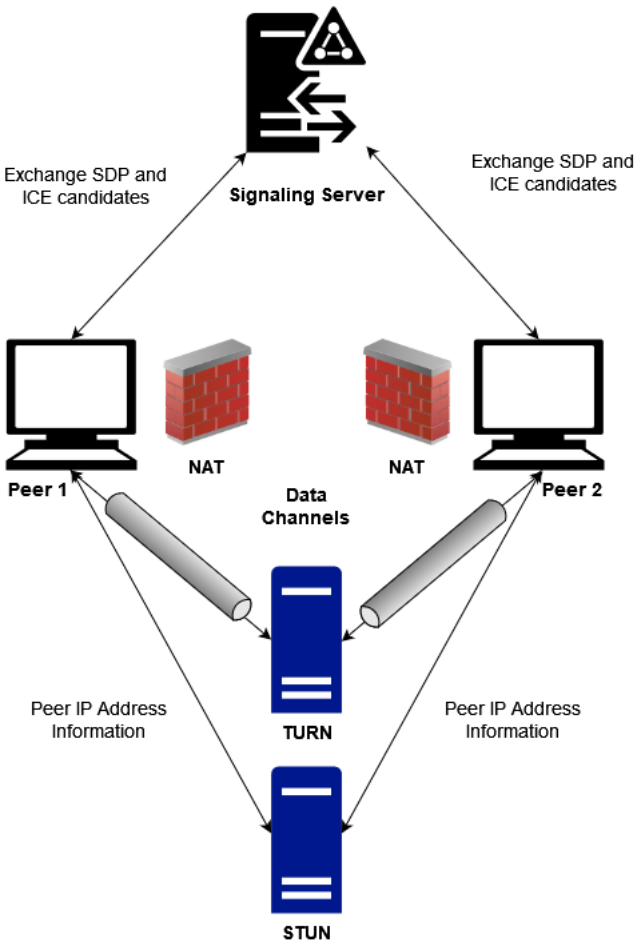

Figure 6: WebRTC Schematic

3 Implementation

The basic metaverse of things provides location independent data exchange between participating IoT devices and humans. From an economic standpoint it is important to keep the focus on the service level and most abandoned software used by individuals to access the internet, because a potential market of the metaverse of things needs to be reach by as many customers as possible. Therefore modern browsers and there supported standards are the main target to grant access to the metaverse. As described in chapter 2.5 WebRTC will enable communication and WebGL will allow a plugin-free render of 3D graphics on an HTML5 web page. The integration of IoT devices in one environment with humans cannot go without the clarification of ownership. That means that a metaverse of things would implement accounts for users that associate ownership, rights and responsibilities with it. The network master system from godot alongside with restricted path access on the website established with Nginx implements clarification of ownership and allows the owner of the UAV to grant or withdraw permission to control and observe. The owner will now be referred to as Master and other clients will now be referred to as Observers. Handling a drone requires consideration of law and insurance. All test with the drone are performed without its ability to lift of for security reasons.

3.1 Webservice

The web service is set up in a generic home network environment. Internet access is provided by A1 Telekom Austria AG through a TECHNICOLOR TG588v router and NAT device. The Raspberry Pi 4 is connected via LAN to the router. All necessary servers an micro services are executed on the raspberry pi 4 and run in their own docker container. To enable the WebRTC there is a signaling server and the open-source coturn Turn server. Two versions of the virtual environment with different permission settings, one certification service that gathers CA certificates from Let's encrypt website and finally, the nginx reverse proxy and python flask server. Flask does provide the website and acts as authentication service, whereas nginx passes traffic to authenticated paths.

From the outside the service website is accessible by any modern browser through the domain `https://www.bennysdrone.at`. Http (Port 80) requests are passed to Https (port 443) by Nginx. The signaling server uses RFC 6455 Websocket protocol to pass ICE candidates and SDP descriptions in JSON strings from and to participants. On an successful connection all communication between participant including the UAV happens through WebRTC data channels. A public available Google STUN server is addressed for ICE candidates gathering and ports 10000 to 11000 are available to relay data packages.

3.1.1 The Website and the Authentication Process

By browsing to the domain mentioned earlier the user will be greeted by a login page. For the sake of this project only two accounts exist. One is the master account and the other one is the observer account. There are two different types of home pages. The master and the observer home page. They only differ in the fact that the placed iframe canvas on the page has a reference to the path .../app/ or .../appM/ respectively. Depending on the credentials put to the website flask will pass the user either to the master page or the

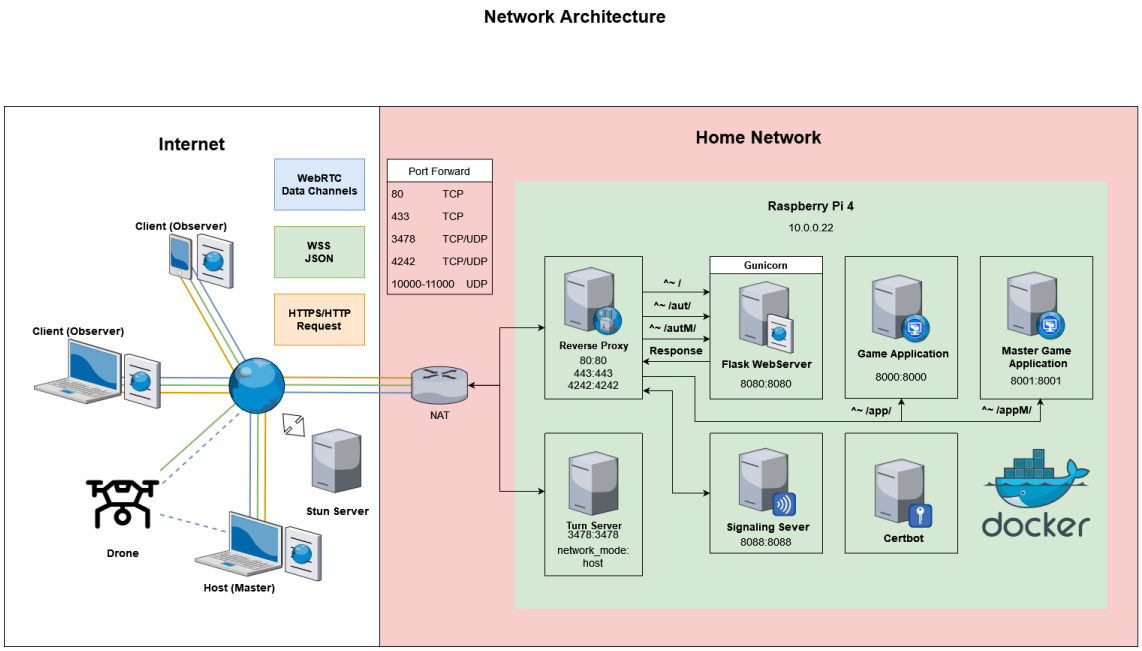

Figure 7: Network architecture overview

observer page. The exported WebGL virtual environment files are then loaded from the referenced iframe path and the output is drawn to the iframe canvas. To prevent users from skipping the authentication Nginx allows only path access to the virtual environment if it gets a http 200 status code from path .../auth/ or .../authM/ respectively. The response is given by the flask web server, which tracks the login state of the current user.

3.2 Virtual Environment

The virtual environment is the graphical interface for the user. GUI elements allow the selection of features, host a session from a lobby as a master or join a session as an observer. Avatars and digital IoT twins are visible there.

3.2.1 Lobby

Before entering the virtual environment a connection with other game instances and IoT devices needs to be initialized. This happens on the lobby screen displayed in figure 9. In case of the user being the master she has the option to host. A user that has the role of an observer will have to wait on the lobby screen until the master is hosting.

3.2.2 Connection

By pressing the host button an introduction message is send to the signaling server at address wss://www.bennysdrone.at:4242. Every message is a JSON string with the fields _type, _to, _data and _id. _type is an identifier that is used for parsing. On introduction to the signaling server, this field will be empty. The signaling server puts the

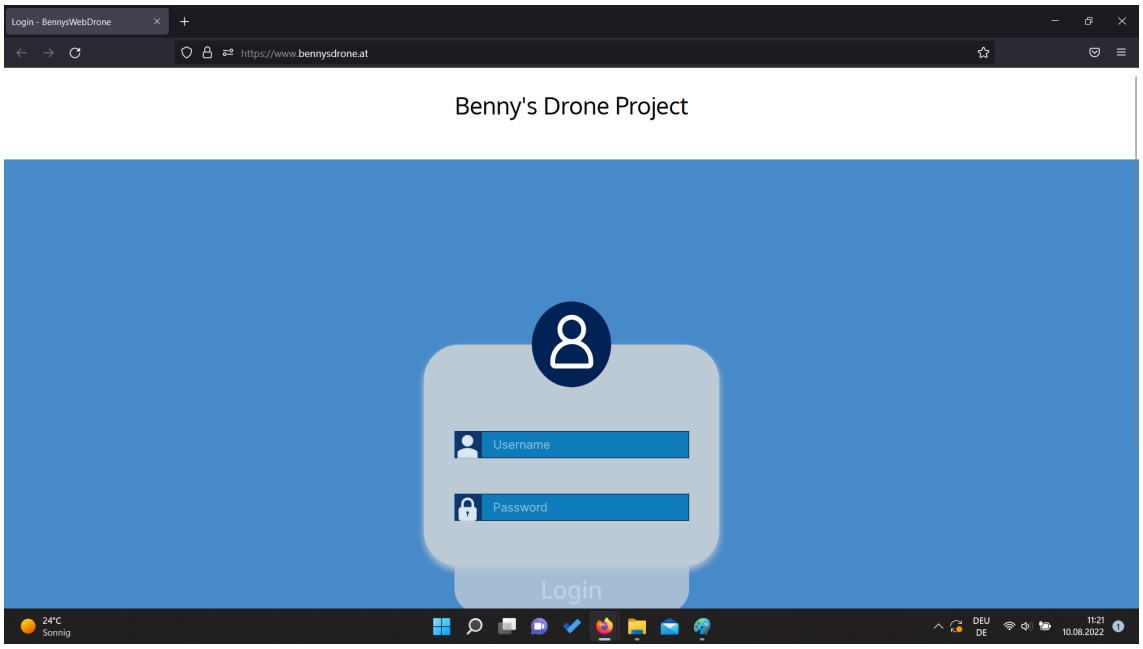

Figure 8: Login page

Figure 9: Iframe cut out from the master home page

calculated uuid in a list. A message is created with the _type field set to "announcement" and the _id field set to the new uuid. This message is send to all registered sockets. On receiving an announcement message with no id assigned, the *NetworkMultiplayerPeer* of the game instance is set to the *WebRTCMultiplayer* class that is initialized with the received unique network id. Receiving the id 1 from the signaling server means that the

game instance has master privileges. The master is identified by a token included in the introduction message. The range of possible network ids is between 1 and 2^{28}-1 (Godot limits the possible network id range from 1 to $2^{31} - 1$). Every game instance has its own list of connected participant ids.

Figure 10 shows the exchanged packages for connection establishment with the contend put in the _data field of the send JSON string. A new registered id is announced to all other connected peers. On receiving a new id from the signaling server, an SDP offer is created and send to the new connected peer. This peer sends an SDP answer back to all ids that send an offer. An exception to this is the UAV. It will only sends an answer to id 1. Any data flow from the drone to other clients nodes have to go through the master node. In Godot 3.2, possible ICE candidates are send one by one after a valid answer is received.

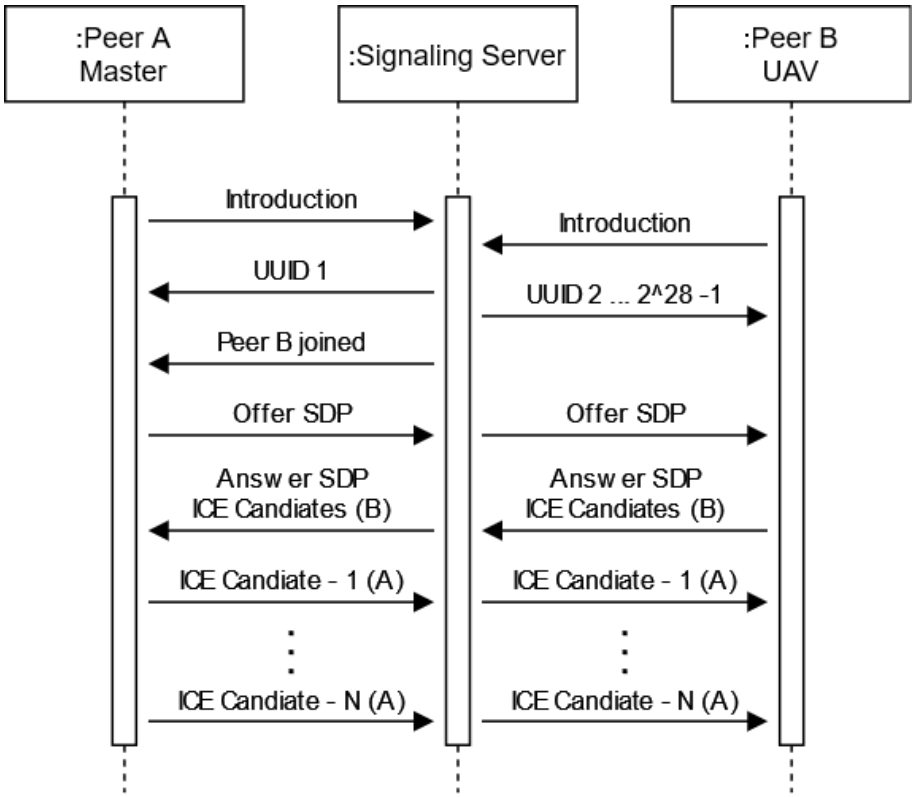

Figure 10: Sequence diagram of the connection process

3.2.3 3 Dimensional Space

The virtual environment consists of a 3D model of the drone and basic ground plane. Master and Observer join around the ground plane and can navigate in 3D space trough mouse control on desktop and virtual joystick control on smartphones. Their avatars are green spheres for Observers, only the Master is displayed in red. Features are accessible

through a radial menu by right mouse click or on touching the screen for more than one second on smartphone devices. They are:

- **Camera**
  On selection an empty texture screen will be displayed right in front of the digital UAV model. If a UAV is connected and camera frames a received from it, they will be transformed in a 2D texture and set as the texture of the texture screen. The refresh rate is bound to godots process frame rate and cannot exceed 60 fps.

- **UAV Control**
  On selection permitted users will switch in first person view of the drone in the virtual environment. the FPV camera screen will be right before them. The UAV can now be controlled by the emulated joystick on mobilephones or by WASDQE keys on a keyboard.

- **Chat box**
  A simple chat box with an input field that allows for an exchange of text messages between the all users.

- **User List**
  Shows a list of all connected users with their chosen pseudonym name. Here the master has the possibility to select which observer can see and control the drone and can remove observers from the session.

- **Return to Lobby**
  Allows to return to the lobby screen. Also accessible with the ESC key.

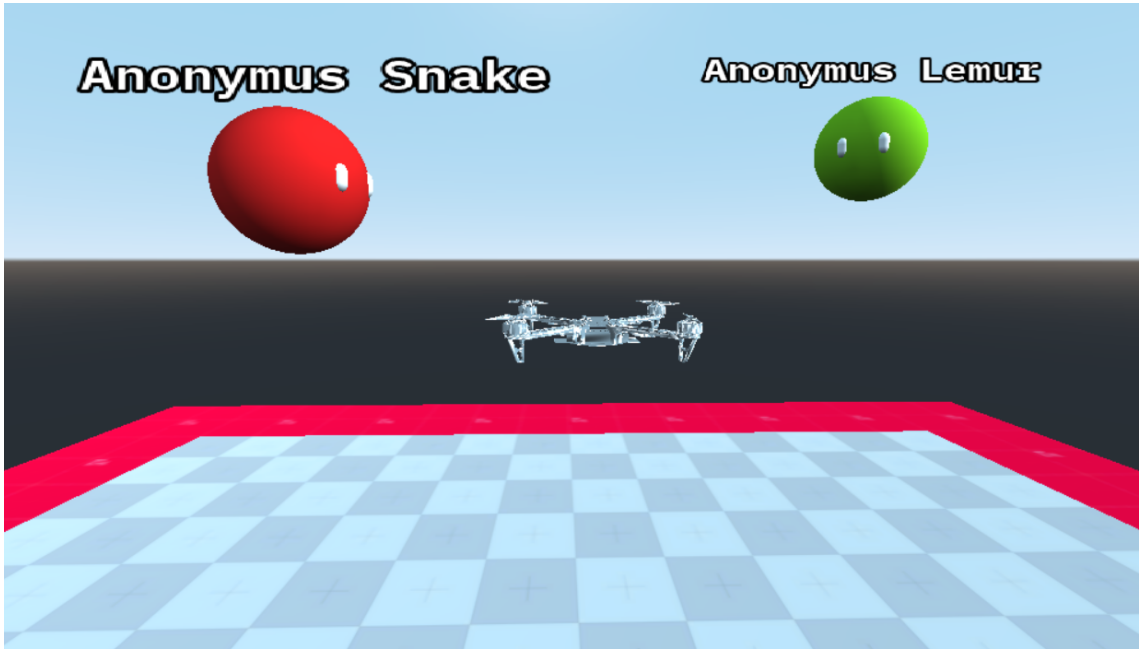

Figure 11: Virtual environment with 3D Model UAV, Host (red) and Client (green) avatar

3.3 UAV Setup

On the UAV there are two software packages that act together in order to control the UAV and communicate with it. One the one hand there is the ardupilot flight control software and one the other hand there is the broker server for connecting to the WebRTC network.

3.3.1 Flight Control Software

Installed on the BeableBoneblue flight controller is the Ardupilot copter software version 3.5. It is set up as *systemd* service that starts in the boot process. Since there is no 5G Module with SIM card connected to the BeagleBoneBlue the connman Wi-Fi services is configured to connect to a mobile hotspot provided by Samsung Galaxy S10. Detailed information on the ardupilot installation process can be found at [17].

3.3.2 The Broker and the MAVLINK Protocol

The broker is a server written in python with the aiortc and asyncio libary. It is responsible to connect to the master once the master is hosting and parse incoming commands from the master and translate them to MAVLINK protocol commands, which are then send to the ardupilot copter software. In addition it runs a webcam service with OpenCV in another process and converts every frame to jpg pictures and encodes them as a base64 string.

MAVLINK is a messaging protocol for communicating with drones [18]. The pymavlink library is used for setting up a connection with ardupilot on port 14550. This port is default for flight control software such as MissionPlaner. After receiving a so called heartbeat signal send every second the connection is established and a data stream is requested with a 10Hz frequency. Only data of type attitude is collected, that includes pitch, roll and yaw data. To send the data to the master, this data is put in a JSON dictionary. Additionally the encoded jpg string is also put in this dictionary and send every 20ms as UDP packet to the master through a WebRTC data channel.

4 Evaluation

4.1 Latency of data packages

To estimate the real time capabilities of the system the currentRoundTripTime property is evaluated. It is a floating-point value indicating the number of seconds it takes for data to be sent by this peer to the remote peer and back over the connection described by a pair of ICE candidates. This property can be viewed in a chrome browser at `chrome://webrtc-internals/`, alongside other useful statistics.

4.1.1 Master and Observer in Local Network

In the first setup two tabs with the MoT are opened in a chrome browser. On on a desktop PC with and windows OS and the other one on an Apple iPad with iOS. On the PC the master hosts a session and on the tablet the observer joins the session. The gathered candidate pair can be seen at figure 12. No NAT needs to be traversed, because the participants are in the same network. The srflx address is an intermediate address selected by the STUN server, which happens to be the IPv4 address of the PC assigned by local network DNS server.

ICE Candidate pair: :55836 <=> 78.154.135.199:63760

▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type
RTCIceCandidatePair_yeVbtsph_dFtowG0s	**succeeded**	**unknown**		**udp**
RTCIceCandidate_yeVbtsph	local-candidate		55836	host
RTCIceCandidate_dFtowG0s	remote-candidate	78.154.135.199	63760	srflx
RTCIceCandidatePair_sKS9qig8_dFtowG0s	waiting	unknown		udp
RTCIceCandidate_sKS9qig8	local-candidate	188.22.188.222	10790	relay(udp)
RTCIceCandidate_dFtowG0s	remote-candidate	78.154.135.199	63760	srflx

Figure 12: Gathered ICE candidates for local connection

In Figure 13 some selected graphs are displayed from the *webrtc-internals* page. Despite from a few peaks, the currentRoundTripTime value stays close to the zero line. This is to be expected if the master and the observer instance are on the same network. The graph *bytesSent* shows a constant rising line over a period of 6 minutes. This is due to the fact that the avatars are constantly sending their spatial information 30 times a second to all established data-channels.

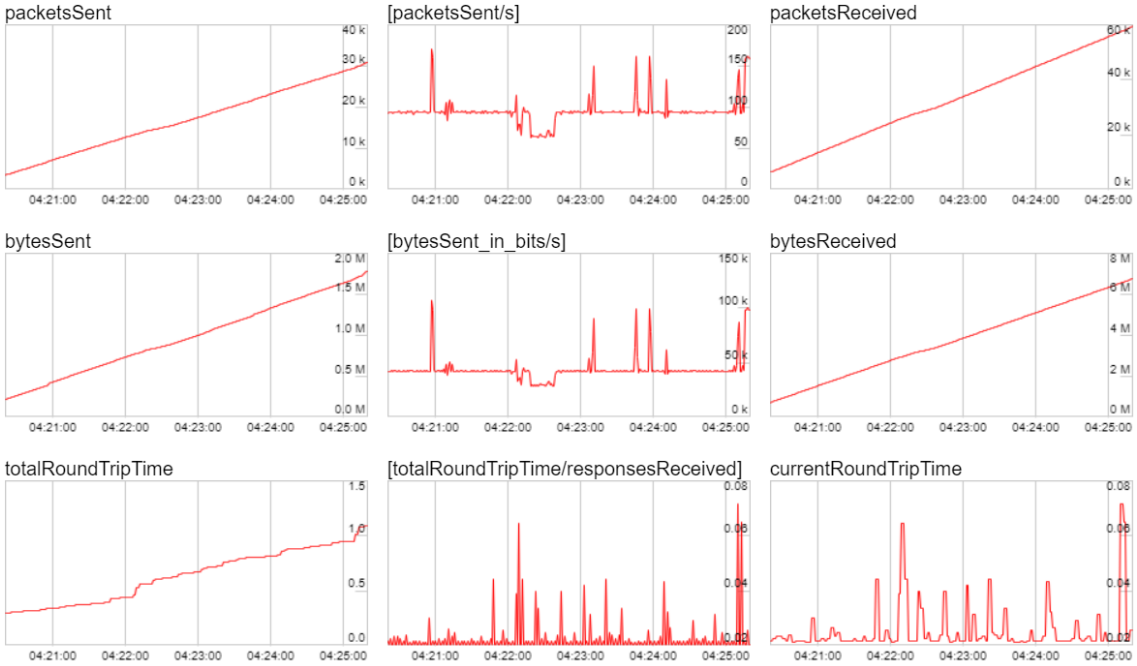

Figure 13: currentRoundTripTime for local connection among other selected statistical graphs

4.2 Master and Observer in separate Networks

If master and observer are not in the same network data traffic is relayed over the TURN server. 188.22.188.222 is the resolved IP current address for `www.bennysdrone.at`. For this session the master can be reached on port 10307 and the observer on port 10074.

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type
RTCIceCandidatePair_fSgVSOFJ_870OoR8w	**succeeded**	**unknown**		**udp**
RTCIceCandidate_fSgVSOFJ	local-candidate	188.22.188.222	10307	relay(udp)
RTCIceCandidate_870OoR8w	remote-candidate		35791	prflx
RTCIceCandidatePair_fSgVSOFJ_zXCR0HuY	waiting	unknown		udp
RTCIceCandidate_fSgVSOFJ	local-candidate	188.22.188.222	10307	relay(udp)
RTCIceCandidate_zXCR0HuY	remote-candidate	188.22.188.222	10074	relay

Figure 14: Gathered ICE candidates for remote connection

In Figure 15 one can see the that the *currentRoundTripTime* value is now significant greater than it was in with all participants in the local Network. Mean is between 100 and 200ms.

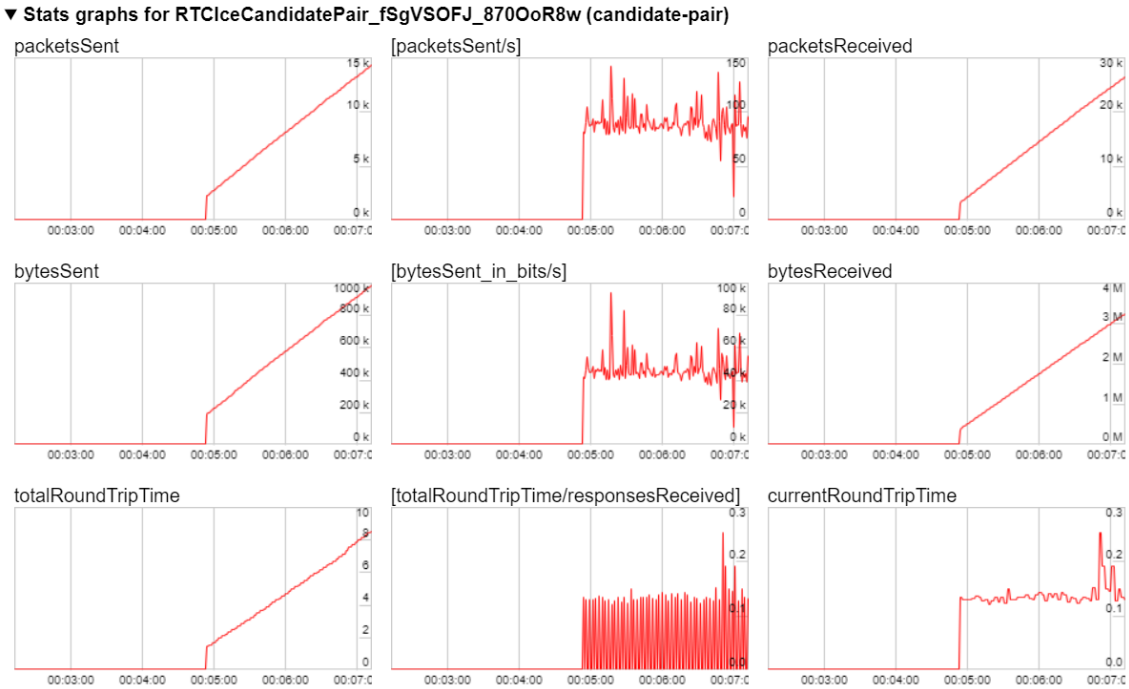

Figure 15: currentRoundTripTime for remote connection among other selected statistical graphs

4.3 Master and UAV in Local Network

For this test the UAV is connected to the master. The *protocol/candidate type* in figure 16 shows only host, because the broker is deployed on the hosting PC. Please find reason in Chapter 5 .

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type
RTCIceCandidatePair_zjmd42cn_zKEYi8pr	**succeeded**	**unknown**		**udp**
RTCIceCandidate_zjmd42cn	local-candidate		56660	host
RTCIceCandidate_zKEYi8pr	remote-candidate	78.154.135.199	56951	host

Figure 16: currentRoundTripTime for remote UAV connection among other selected statistical graphs

In figure 17 we see that the *currentRoundTripTime* value is always below 0.05s. The amaount of *byteSent* is now significantly higher, because the broker is sending FPV camera

frames, roll, pitch and yaw data received via MAVLINK to the master.

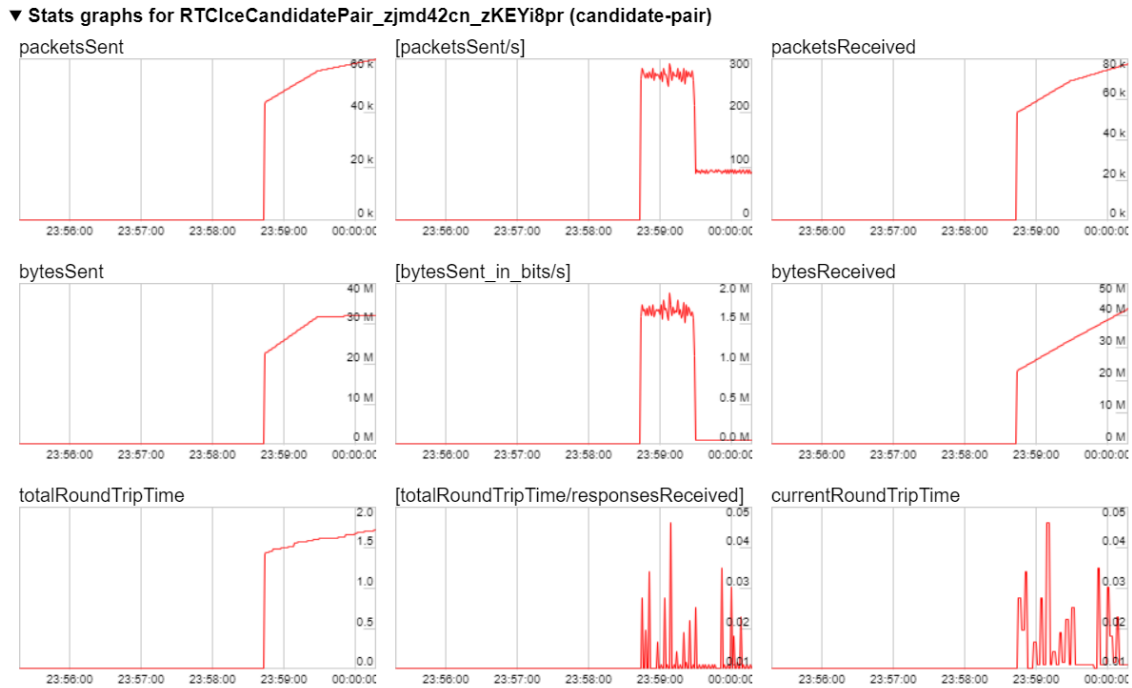

Figure 17: Gathered ICE candidates for local UAV connection

4.4 Master and UAV in separate Networks

The candidates in figure 19 are captured from a Samsung Galaxy S10, which takes the role of the master device and is connected to the Internet via 5G. The number of candidates is higher and only a selection is visible. All data traffic is passes the TURN server.

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type
RTCIceCandidatePair_svc3x7mY_vVQHMSJV	**succeeded**	**unknown**		**udp**
RTCIceCandidate_svc3x7mY	local-candidate	89.144.195.158	6034	prflx
RTCIceCandidate_vVQHMSJV	remote-candidate	78.154.135.199	49247	host
RTCIceCandidatePair_HTyjNEVa_LiCGQ0gi	in-progress	unknown		udp
RTCIceCandidate_HTyjNEVa	local-candidate	10.164.215.129	38221	host

Figure 18: Gathered ICE candidates for local UAV connection, table beginning

RTCIceCandidatePair_PjWC+P8M_vVQHMSJV	waiting	unknown		udp
RTCIceCandidate_PjWC+P8M	local-candidate	188.22.188.222	10381	relay(udp)
RTCIceCandidate_vVQHMSJV	remote-candidate	78.154.135.199	49247	host
RTCIceCandidatePair_PjWC+P8M_vn1Tgt+h	waiting	unknown		udp
RTCIceCandidate_PjWC+P8M	local-candidate	188.22.188.222	10381	relay(udp)
RTCIceCandidate_vn1Tgt+h	remote-candidate	192.168.56.1	49246	host

Figure 19: Gathered ICE candidates for local UAV connection, table end

In figure 20 the *currentRoundTripTime* value remains between 100ms and 200ms after an initial peak. These are similar measurements compared to the results of chapter 4.2.

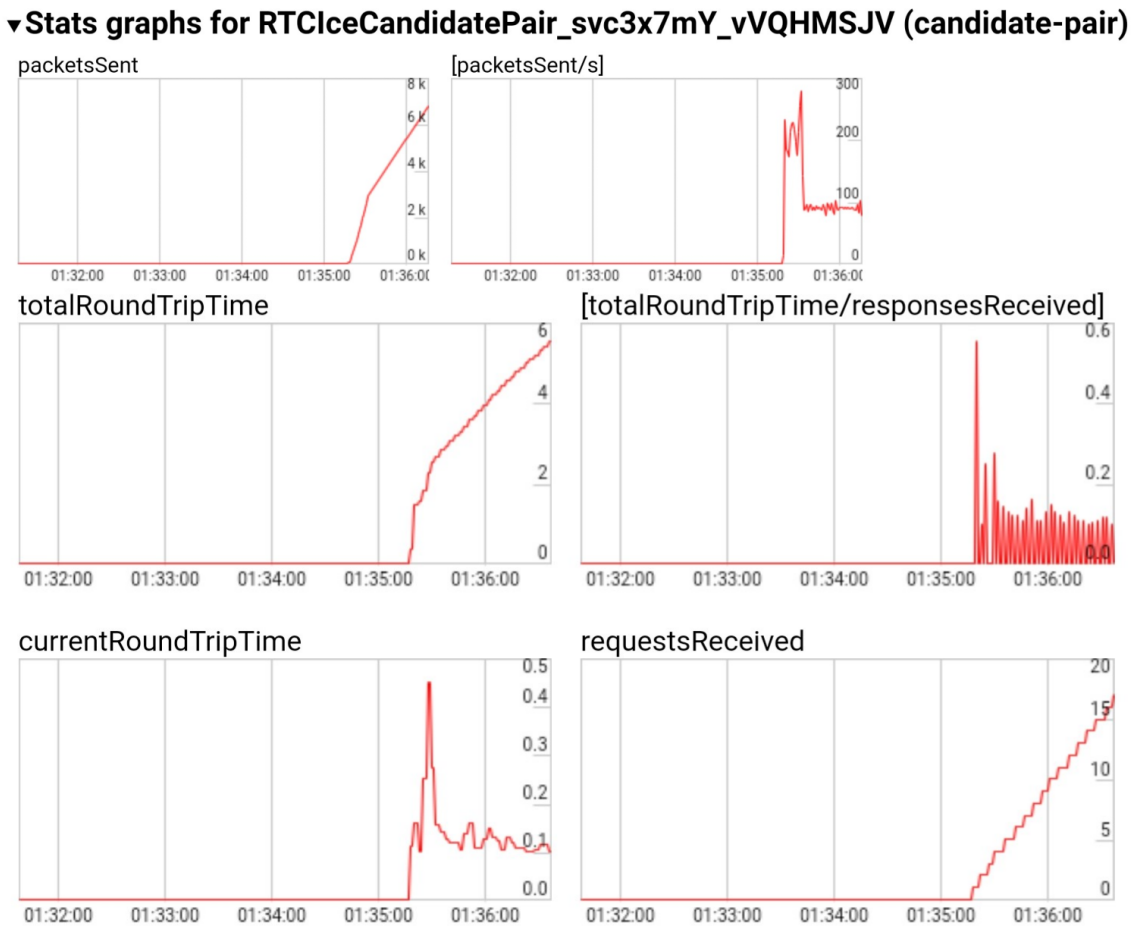

Figure 20: currentRoundTripTime for remote UAV connection among other selected statistical graphs

5 Limitations

The MoT project purposed in this thesis is far from completion at its current state. Some crucial elements of an MoT descried in Chapter 2.1 are missing, such as persistence and its own an economic market. The virtual environment does not permanently change on data input and no purchase or sell functionalities are implemented on the website. Minor bugs in the GUI elements are present. Tests with joystick control where conducted without mounted propellers. Testing of flight capabilities requires insurance and indoor or outdoor test facilities and could not be evaluated.

Due to problems with dependencies on the BeagleBoneBlue regarding the pymavlink library the broker was actually deployed on a laptop with Windows OS for all tests conducted in Chapter 4 . This resembles the deployment of the broker on an edge node, which was not originally anticipated. As a result, the FPV camera is replaced by the webcam of the laptop. This explains the *Protocol/candidate type* to be host for the local- and remote-candidate. MAVLINK commands are send to the port 14550 of the laptop, which is also the default port ArduPilots control software *MissionPlaner* [19] is listening to. This means that the drone is only a indirect part of the WebRTC network.

6 Future Work

This chapter describes which changes and extensions to the current project need to be made to reach the goal of a fully functional MoT. Feature updates are discussed as well as how to ensure interoperability between many IoT devices.

6.1 Fixes and Changes

1. A rework of the user interface needs to be done. If an observer goes back to the lobby he cannot continue the application, but has to reload the website. On mobile devices the chat box and other GUI elements are hard to read and to use. A separate GUI just for mobile devices needs to be developed.

2. ArduPilot does not allow to execute commands such as MAV_CMD_NAV_TAKEOFF as long as there is no latitude and longitude information. Guided flight by waypoints requires localization of the drone. An implementation of an GPS sensor for outdoor and ultra wide band sensors for indoor real-time localization applications would be required.

3. The drone does not update the virtual environment, beside its digital twin. To change that, proximity sensors can be installed on the drone sending data of detected obstacles to the WebRTC instances to render them in the virtual environment.

4. A save system needs to be implemented that uploads any change in the virtual environment to an database. On rejoin to the network environment data can be loaded from the database to ensure persistence in the virtual environment.

5. Increasing the number of IoT devices opens the possibility that multiple can explore there surroundings and update the virtual environment simultaneously.

6.2 Interoperability with the help of Web of Things

In order to ensure the interoperability of MoT with different IoT devices standardized metadata based on the *Web of Tings* (WoT) an W3C initiative is necessary. If every IoT device gets its own WoT Thing Description [20] and the MoT has an API that allow access to the IoTs metadata that would significantly reduce effort to introduce new IoT devices to the MoT. For example, a company that produces drones could be asked to provide an WoT Thing Description for their products and the metadata of their drones is then projected to a digital twin in the MoT. The WoT API would take over the tasks of data exchange the broker is currently providing.

7 Conclusion

Although a the creation of a complete MoT could not be achieved it could be shown that integration of digital twins representing robots and IoT devices on the one side and humans represented by avatars on the other side in one virtual is feasible with open source technology. With the established WebRTC standard data transfer is achieved between participants with reasonable low time delays. Immediate responsive control of an IoT device by a user might not be possible with the current performance, control of an autonomous acting IoT device seems more likely. Flight tests need to be performed to test these assumption as well as optimization of the network architecture and integration of WoT approaches in follow up projects.
Open source software such as the Godot game engine and essential libraries for Python make the development of an metaverse an possibility even for a single person. A monolithic structure could be avoided by splitting the metaverse of things into different micro services. Still developing a metaverse remains no easy task. It is too early to draw an conclusion on the question if a metaverse is a valid and working service option for IoT in industrial or non-industrial sectors, but the prospect of a platform combining social media elements to reach millions of people with open standards reaching millions of smart objects is a topic worthwhile to venture further into.

References

[1] Linden Lab. *Secound Life*. Accessed 30.03.2022. URL: `https://www.lindenlab.com/about`.

[2] Josip Almasi. *How to build an open source metaverse*. Accessed 30.03.2022. URL: `https://opensource.com/article/22/1/open-source-metaverse`.

[3] Meta. *Meta*. Accessed 30.03.2022. URL: `https://about.facebook.com/de/technologies/`.

[4] S. Narasimha Swamy and Solomon Raju Kota. "An Empirical Study on System Level Aspects of Internet of Things (IoT)". In: *IEEE Access* 8 (2020), pp. 188082–188134. DOI: `10.1109/ACCESS.2020.3029847`.

[5] Amazon Staff. Accessed 03.09.2022. URL: `https://www.aboutamazon.com/news/transportation/amazon-prime-air-prepares-for-drone-deliveries`.

[6] SPH Engineering. Accessed 03.09.2022. URL: `https://www.ugcs.com/`.

[7] THE KHRONOS GROUP INC. Accessed 03.09.2022. URL: https://metaverse-standards.org/.

[8] Guang Ji et al. "Digital Twin Modeling Method for Individual Combat Quadrotor UAV". In: *2021 IEEE 1st International Conference on Digital Twins and Parallel Intelligence (DTPI)*. 2021, pp. 1–4. DOI: 10.1109/DTPI52967.2021.9540131.

[9] Juan Linietsky. Accessed 04.06.2022. URL: https://godotengine.org/article/retrospective-and-future.

[10] Ariel Manzur Juan Linietsky and contributors. Accessed 04.06.2022. URL: https://godotengine.org/.

[11] Linietsky Juan, Manzur Ariel, and the Godot community. 04.07.2022. URL: https://docs.godotengine.org/en/stable/about/list_of_features.html#features.

[12] Lee Salzman. Accessed 02.04.2022. URL: http://enet.bespin.org/.

[13] Apple et al. Accessed 02.04.2022. URL: https://w3c.github.io/webrtc-pc/#sotd.

[14] Internet Engineering Task Force. Accessed 02.04.2022. URL: https://datatracker.ietf.org/doc/html/rfc6455.

[15] Google WebRTC-Team. Accessed 09.08.2022. URL: https://webrtc.org/.

[16] Accessed 09.08.2022. URL: https://www.w3.org/TR/webrtc/.

[17] ArduPilot Dev Team. Accessed 26.08.2022. URL: https://ardupilot.org/copter/docs/common-beagle-bone-blue.html?highlight=beagleboneblue#connect-bbblue-to-the-internet.

[18] Lorenz Meier et al. Accessed 26.08.2022. URL: https://mavlink.io/en/.

[19] ArduPilot Dev Team. Accessed 05.09.2022. URL: https://ardupilot.org/planner/.

[20] W3C. Accessed 05.09.2022. URL: https://www.w3.org/WoT/documentation/.